



AI EMPIRE ACADEMY – GIÁO TRÌNH CHUYÊN ĐỀ

BỘ NHỚ HỘI THOẠI VỚI LANGGRAPH

Từ message và thread đến context dài, Store và
FastAPI

Giáo trình thực hành từ nền tảng đến chatbot nhiều cuộc hội thoại

Biên soạn: Mr. Tran Minh Nhat

Đơn vị: AI Empire Academy

Liên hệ: 0976059814

Website: aiempireacademy.com.vn

Phiên bản học thuật 2026

Mục lục

Lời mở đầu	vii
Cách học và kế hoạch 10 buổi	ix
1 MODEL KHÔNG TỰ NHỚ	1
1.1 Một request là một phong bì kín	1
1.2 Ba lớp thường bị gọi chung là “lịch sử”	1
1.3 Xây memory thủ công trước	1
1.4 Vì sao danh sách thuần chưa đủ cho backend?	2
2 STATE, NODE, EDGE VÀ GRAPH	4
2.1 State là hợp đồng dữ liệu	4
2.2 Node là hàm Python	4
2.3 Edge định nghĩa thứ tự	4
2.4 Compile không đồng nghĩa persistence	5
2.5 Đọc graph bằng năm câu hỏi	5
3 SHORT-TERM MEMORY THEO THREAD	7
3.1 Thêm InMemorySaver	7
3.2 Dòng thời gian của hai thread	8
3.3 Chỉ gửi message mới	8
3.4 InMemorySaver dùng đúng chỗ	9
3.5 Đọc state để debug	9
4 QUẢN LÝ CONTEXT DÀI	10
4.1 Vì sao context dài không miễn phí?	10
4.2 Trim là tạo một view ngắn	10
4.3 Không cắt message một cách mù quáng	11
4.4 Phân bổ context có chủ đích	11
5 RUNNING SUMMARY	13
5.1 State mở rộng	13
5.2 Chu trình tóm tắt tăng dần	13
5.3 Summary tốt phải giữ điều gì?	13
5.4 Ví dụ compact quyết định	14
5.5 Khi nào summary chạy?	15

6	LONG-TERM MEMORY THEO USER	16
6.1	Namespace và key	16
6.2	Memory nào đáng lưu?	17
6.3	Kết hợp thread và user	17
6.4	Key-value hay semantic search?	19
7	PERSISTENCE VÀ RANH GIỚI PRODUCTION	20
7.1	Chọn backend theo giai đoạn	20
7.2	Async backend đi với async graph	20
7.3	Ownership là invariant tầng ứng dụng	21
7.4	Retention, xóa và quyền được quên	21
7.5	Checklist vận hành	21
8	TÍCH HỢP VÀO LLM GATEWAY	23
8.1	Hợp đồng API đề xuất	23
8.2	API chạy được với model quyết định	23
8.3	Adapter model thật	24
8.4	System prompt, profile và context	25
8.5	Migration theo lát mỏng	26
9	KIỂM THỬ, DEBUG VÀ QUAN SÁT	27
9.1	Bốn invariant cốt lõi	27
9.2	Test API, không chỉ test graph	28
9.3	Failure lab	29
9.4	Quan sát gì trong log và metric?	29
9.5	Đánh giá chất lượng summary	29
10	DỰ ÁN CUỐI KHÓA: CHATBOT NHIỀU CUỘC HỘI THOẠI	31
10.1	Đề bài	31
10.2	Yêu cầu chức năng	31
10.3	Yêu cầu phi chức năng	31
10.4	Kiến trúc tham chiếu	32
10.5	Kế hoạch thực hiện theo lát	32
10.6	Demo 5 phút bắt buộc	32
10.7	Definition of Done	32
	Kết luận	34
A	Cài đặt và lệnh chạy	35
A.1	Cài môi trường	35
A.2	Provider thật	35
A.3	Build PDF	35
A.4	Fallback	36
B	Bài tập tổng hợp và rubric	37

B.1	Bài tập cấp 1: mô hình tinh thần	37
B.2	Bài tập cấp 2: LangGraph memory	37
B.3	Bài tập cấp 3: production	37
B.4	Rubric dự án cuối khóa	38
B.5	Thang phản hồi bốn mức	38
C	Thuật ngữ nhanh	39
	Tài liệu tham khảo	40

Danh sách hình vẽ

1.1	Hai request độc lập khi ứng dụng không gửi lại lịch sử.	1
2.1	Graph hội thoại nhỏ nhất chưa có persistence.	4
3.1	Cùng graph và checkpoint, state được phân vùng bởi thread.	8
4.1	Trim tạo view cho model nhưng có thể giữ storage đầy đủ.	11
5.1	Running summary chỉ chạy khi history vượt ngưỡng.	13
6.1	Nhiều thread của cùng user truy cập một namespace memory.	19
8.1	Ranh giới thành phần trong LLM Gateway có memory.	26
10.1	Kiến trúc tham chiếu cho dự án cuối khóa.	32

Danh sách bảng

1	Kế hoạch học theo từng buổi 90 phút	ix
1.1	Ba lớp history trong một ứng dụng chat	1
4.1	Ví dụ phân bổ context; số thực tế tùy model và use case	11
6.1	Checkpoint và Store	16
7.1	Lộ trình backend cho memory	20
9.1	Tín hiệu vận hành không cần log toàn bộ nội dung chat	29
10.1	Các mốc dự án có thể kiểm chứng	32
B.1	Rubric 100 điểm cho chatbot có memory	38
C.1	Thuật ngữ cốt lõi	39

Lời mở đầu

Một giao diện có bong bóng tin nhắn chưa chắc đã là một chatbot có trí nhớ. Model ngôn ngữ chỉ thấy dữ liệu nằm trong request hiện tại; nếu ứng dụng không gửi lại thông tin cũ, câu hỏi “Tôi tên gì?” không có cách nào liên hệ với câu “Tôi tên Nhật” ở request trước. Từ quan sát nhỏ này xuất hiện cả một lớp bài toán hệ thống: lưu message ở đâu, phân tách nhiều cuộc trò chuyện thế nào, đưa bao nhiêu lịch sử vào context, tóm tắt phần cũ ra sao, và thông tin nào được phép sống lâu hơn một thread.

Giáo trình này xây mô hình tinh thần trước rồi mới thêm framework. Học viên bắt đầu bằng một danh sách Python thuần, chuyển nó thành state của LangGraph, thêm checkpoint, quan sát hai `thread_id` độc lập, rồi mới học trimming, running summary và long-term Store. Phần cuối ghép các mảnh ấy vào FastAPI và thiết kế API cho một LLM Gateway nhiều provider.

Định nghĩa: Phạm vi

Cuốn sách tập trung vào bộ nhớ hội thoại và context engineering. Qdrant, RAG và agent gọi tool chỉ được nhắc ở ranh giới cần phân biệt; chúng không phải điều kiện để hoàn thành các lab.

Thực giác: Nguyên tắc xuyên suốt

Model không sở hữu trí nhớ. Ứng dụng sở hữu dữ liệu; LangGraph giúp tổ chức state, checkpoint và luồng cập nhật; model chỉ nhận một lát context do ứng dụng lựa chọn ở thời điểm gọi.

Mọi ví dụ cốt lõi dùng một “model quyết định” bằng quy tắc. Nhờ vậy học viên kiểm tra được memory mà không phụ thuộc mạng, API key hay độ ngẫu nhiên của LLM. Khi cơ chế đã đúng, ta thay model giả bằng DeepSeek hoặc Gemini mà không đổi invariant của hội thoại.

Đối tượng và điều kiện đầu vào

Tài liệu dành cho người đã biết Python cơ bản, dictionary, list, function và cách chạy một ứng dụng FastAPI; không giả định đã dùng LangChain hay LangGraph. Python 3.11 trở lên và công cụ uv là cấu hình đề xuất. LangGraph hiện yêu cầu Python 3.10 trở lên và được cài độc lập với các gói provider (LangChain 2026c).

Kết quả đầu ra quan sát được

Sau khi hoàn thành, học viên có thể:

1. giải thích vì sao UI history khác model context và persistent history;
2. dựng graph `START -> call_model -> END`;
3. dùng checkpointer và `thread_id` để tách cuộc trò chuyện;
4. chọn trim hoặc summary dựa trên chi phí và yêu cầu giữ thông tin;
5. dùng `user_id` và Store cho memory xuyên thread;
6. thiết kế API chống trộn dữ liệu giữa người dùng;
7. viết test chứng minh các invariant trên.

Cách học và kế hoạch 10 buổi

Điểm tự kiểm tra: Nhịp học đề xuất

Mỗi chương theo vòng lặp: dự đoán trước khi chạy; chạy ví dụ nhỏ nhất; vẽ lại state; cố ý dùng sai `thread_id`; chạy test; cuối cùng áp dụng vào LLM Gateway. Học viên nên chủ động ít nhất một nửa thời lượng thay vì chỉ xem giảng viên gõ code.

Bảng 1: Kế hoạch học theo từng buổi 90 phút

Buổi	Trọng tâm	Thực hành	Minh chứng hoàn thành
1	Model context và ba lớp history	Python thuần	Model giả trả đúng tên khi có history.
2	State, node, edge, graph	Graph nhỏ nhất	Vẽ và chạy được graph một node.
3	Checkpoint, thread, checkpoint	Hai thread	Nhật và An không bị trộn tên.
4	Context window và token budget	Trim	Chứng minh storage đầy đủ nhưng model input ngắn.
5	Running summary	Compact history	Summary giữ quyết định, message gần vẫn nguyên văn.
6	Store và user memory	Hai user, nhiều thread	Memory đi qua thread nhưng không đi qua user.
7	SQLite/PostgreSQL	Thiết kế persistence	Chọn backend và mô tả vòng đời dữ liệu.
8	FastAPI/Streamlit	Conversation API	Tạo, chat, đọc và xóa một thread.
9	Test, log, privacy	Failure lab	Test ownership và test restart.
10	Dự án cuối khóa	LLM Gateway	Demo end-to-end và báo cáo trade-off.

Cấu trúc một buổi 90 phút

- **0–8 phút:** kiểm tra mục tiêu và dự đoán output.
- **8–20 phút:** mô hình tinh thần bằng một ví dụ cụ thể.
- **20–35 phút:** giảng viên demo và nói rõ state thay đổi ở đâu.
- **35–55 phút:** thực hành có hướng dẫn, gồm một lỗi cố ý.

- **55–75 phút:** học viên tự áp dụng vào biến thể gần dự án thật.
- **75–85 phút:** chạy test, phản hồi theo rubric, sửa một lần.
- **85–90 phút:** chốt artifact và bài tập về nhà.

Cách đọc code

Mỗi listing quan trọng trong sách lấy từ thư mục `code/`. Trước khi chạy, hãy trả lời bốn câu: input mới là gì; state cũ nằm ở đâu; hàm nào thay state; output nào được lưu lại. Nếu chưa trả lời được, đừng vội đổi sang model thật.

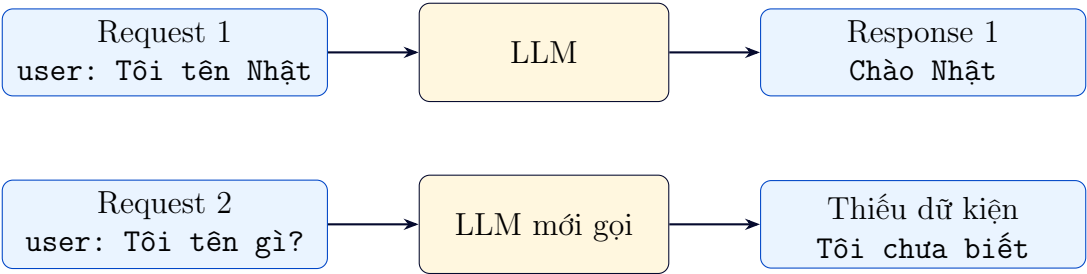
Cảnh báo: Fallback khi không có API key

Không bỏ qua lab. Model quyết định trong tài liệu cố ý không cần API. Memory là thuộc tính của ứng dụng; ta có thể kiểm thử chính xác mà không cần một LLM sinh văn bản.

1. MODEL KHÔNG TỰ NHỚ

1.1. Một request là một phong bì kín

Hãy coi mỗi lần gọi model là gửi một phong bì. Model chỉ đọc giấy nằm trong phong bì đó. Request đầu chứa “Tôi tên Nhật” không tự xuất hiện trong request sau chứa “Tôi tên gì?”. Nhà cung cấp có thể có API hội thoại riêng, nhưng invariant ở tầng ứng dụng vẫn là: phải có một định danh và một cơ chế khôi phục context.



Hình 1.1: Hai request độc lập khi ứng dụng không gửi lại lịch sử.

Nguồn: Tác giả tự dựng.

1.2. Ba lớp thường bị gọi chung là “lịch sử”

Bảng 1.1: Ba lớp history trong một ứng dụng chat

Lớp	Mục đích	Ví dụ
UI history	Vẽ lại bong bóng cho người dùng	<code>st.session_state.messages</code>
Persistent history	Khôi phục sau request hoặc restart	checkpoint, SQLite, PostgreSQL
Model context	Lát dữ liệu thực sự gửi vào LLM	system prompt + summary + message gần

Một lỗi phổ biến là UI có đủ bong bóng nên nhóm phát triển tưởng model đã có context. Lỗi ngược lại là database lưu đầy đủ nên mỗi request gửi toàn bộ lịch sử, khiến chi phí và độ trễ tăng dần.

1.3. Xây memory thủ công trước

Mã nguồn 1.1: Giữ lịch sử bằng Python thuần

```

"""Minimal example: the application owns its conversation history."""

from dataclasses import dataclass, field
import sys

Message = dict[str, str]

def fake_model(messages: list[Message]) -> str:
    """A deterministic model keeps the example independent from API keys."""
    latest = messages[-1]["content"].lower()
    if "t\u00f4i t\u00e2n g\u00ec" in latest:
        for message in reversed(messages[:-1]):
            content = message["content"]
            marker = "T\u00f4i t\u00e2n "
            if message["role"] == "user" and marker in content:
                return f"B\u1ea1n t\u00e2n {content.split(marker, 1)[1].rstrip('.')}.replace(" .", ".")
            return "T\u00f4i ch\u01b0a bi\u1ebft t\u00e2n b\u1ea1n."
        return "T\u00f4i \u0111\u00e3 nh\u1eadn \u0111\u01b0\u1ee3c tin nh\u1eaafn."

    @dataclass
    class ManualConversation:
        messages: list[Message] = field(default_factory=list)

        def ask(self, text: str) -> str:
            self.messages.append({"role": "user", "content": text})
            answer = fake_model(self.messages)
            self.messages.append({"role": "assistant", "content": answer})
            return answer

    if __name__ == "__main__":
        if hasattr(sys.stdout, "reconfigure"):
            sys.stdout.reconfigure(encoding="utf-8")
        chat = ManualConversation()
        print(chat.ask("T\u00f4i t\u00e2n Nh\u1eadt."))
        print(chat.ask("T\u00f4i t\u00e2n g\u00ec?"))

```

Đối tượng `ManualConversation` sở hữu danh sách message. Mỗi lượt `ask` phải thực hiện đúng thứ tự: thêm human message; gọi model với danh sách hiện tại; thêm assistant message. Đây là cơ chế nền tảng mà framework sẽ tổ chức tốt hơn, không phải phép màu mới.

Ví dụ: Dự đoán trước khi chạy

Nếu tạo hai instance `ManualConversation`, nói tên Nhật ở instance thứ nhất và hỏi tên ở instance thứ hai, câu trả lời đúng phải là “Tôi chưa biết tên bạn”. Mỗi object hiện đại diện một conversation độc lập.

1.4. Vì sao danh sách thuần chưa đủ cho backend?

Trong một script, object còn sống giữa hai dòng lệnh. Trong FastAPI có nhiều request đồng thời, nhiều user, nhiều worker và restart. Ta phải giải quyết:

- khóa nào tìm đúng conversation;

- ai sở hữu conversation đó;
- cập nhật đồng thời có bị mất message không;
- dữ liệu có sống qua restart không;
- phần nào được đưa vào model khi history quá dài.

Định nghĩa: Hai định danh

`thread_id` nhận diện một cuộc trò chuyện. `user_id` nhận diện một người dùng. Một user có thể có nhiều thread; một thread trong ứng dụng chat thông thường chỉ thuộc một user.

Điểm tự kiểm tra: Lab 15 phút

Chạy `manual_history.py`; sau đó tạo `chat_a` và `chat_b`. Dự đoán rồi kiểm tra bốn output khi A tên Nhật, B tên An và mỗi bên hỏi lại tên. Minh chứng đạt: không có câu trả lời nào trộn tên giữa hai object.

2. STATE, NODE, EDGE VÀ GRAPH

LangGraph là hạ tầng mức thấp cho workflow có trạng thái và chạy lâu; nó không tự quyết định prompt hay kiến trúc ứng dụng (LangChain 2026d). Vì thế cách học tốt nhất là dựng graph nhỏ nhất, quan sát input/output, rồi mới thêm memory.

2.1. State là hợp đồng dữ liệu

State là ảnh chụp dữ liệu mà các node cùng đọc và cập nhật. Với chatbot, state tối thiểu có khóa `messages`. `MessagesState` cung cấp reducer để message mới được nối vào danh sách thay vì ghi đè toàn bộ.

Định nghĩa: Reducer

Reducer là quy tắc hợp nhất giá trị node trả về với state hiện tại. Với `messages`, ý nghĩa mong muốn thường là “append message mới”; với một cờ boolean, ý nghĩa có thể là “ghi đè”.

2.2. Node là hàm Python

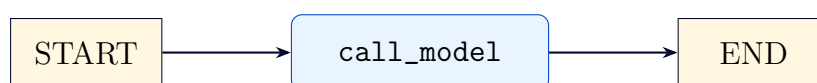
Node nhận state và trả về phần cập nhật:

Mã nguồn 2.1: Hợp đồng của một model node

```
def call_model(state: MessagesState) -> dict:
    latest = state["messages"][-1].content
    response = make_response(latest)
    return {"messages": [response]}
```

Node không cần trả toàn bộ state. Nó chỉ trả delta mà mình tạo. Đây là chi tiết giúp graph có thể quan sát từng bước và checkpoint state.

2.3. Edge định nghĩa thứ tự



Hình 2.1: Graph hội thoại nhỏ nhất chưa có persistence.

Nguồn: Tác giả tự dựng theo LangGraph Graph API.

Mã nguồn 2.2: Graph chạy được không cần API key

```
"""The smallest graph: START -> call_model -> END."""

import sys

from langchain_core.messages import AIMessage
from langgraph.graph import END, START, MessagesState, StateGraph

def call_model(state: MessagesState) -> dict:
    latest = state["messages"][-1].content
    return {"messages": [AIMessage(content=f"\u0110\u00e3 nh\u01eadn: {latest}")] }

builder = StateGraph(MessagesState)
builder.add_node("call_model", call_model)
builder.add_edge(START, "call_model")
builder.add_edge("call_model", END)
graph = builder.compile()

if __name__ == "__main__":
    if hasattr(sys.stdout, "reconfigure"):
        sys.stdout.reconfigure(encoding="utf-8")
    result = graph.invoke(
        {"messages": [{"role": "user", "content": "Xin ch\u00e0o LangGraph"}]}
    )
    print(result["messages"][-1].content)
```

Khi invoke, input human message tạo state ban đầu. Node thêm một AI message. Result cuối chứa cả hai message. Nhưng graph này chưa có checkpoint: lần invoke sau bắt đầu với state mới nếu caller không tự gửi lịch sử.

2.4. Compile không đồng nghĩa persistence

`builder.compile()` kiểm tra và đóng gói graph để chạy. Muốn lưu state theo thread, ta phải truyền một checkpoint khi compile. Phân biệt hai bước:

Mã nguồn 2.3: Cùng graph, khác năng lực lưu state

```
stateless_graph = builder.compile()
stateful_graph = builder.compile(checkpointer=checkpointer)
```

Cảnh báo: Nhầm lẫn thường gặp

`MessagesState` mô tả cách state chứa message; nó không tự lưu message qua hai lần invoke. Checkpointer mới là thành phần persistence theo thread.

2.5. Đọc graph bằng năm câu hỏi

Khi gặp code LangGraph, hãy lần lượt hỏi:

1. State schema có những khóa nào?

2. Mỗi node đọc khóa nào và trả delta nào?
3. Edge nào quyết định node tiếp theo?
4. Graph compile với checkpointer hoặc store nào?
5. Khi invoke, config và runtime context mang định danh nào?

Điểm tự kiểm tra: Lab 20 phút

Thêm node `normalize_input` trước `call_model`. Node này loại khoảng trắng đầu cuối của human message. Vẽ lại graph, dự đoán state sau từng node và viết một assert cho output. Không thêm agent hoặc tool ở lab này.

3. SHORT-TERM MEMORY THEO THREAD

Short-term memory là state gắn với một thread. Khi graph compile với checkpoint, state được ghi thành checkpoint ở các bước thực thi và có thể khôi phục ở lần gọi sau. `thread_id` là khóa để checkpointer tìm các checkpoint ấy (LangChain 2026e).

3.1. Thêm InMemorySaver

Mã nguồn 3.1: Ba dòng tạo short-term memory

```
from langgraph.checkpoint.memory import InMemorySaver

checkpointer = InMemorySaver()
graph = builder.compile(checkpointer=checkpointer)
config = {"configurable": {"thread_id": "thread-a"}}
```

Mỗi lần invoke cùng config, LangGraph nạp state của `thread-a`, hợp nhất input message mới, chạy node và lưu state mới. Tài liệu LangChain gọi đây là thread-level persistence (LangChain 2026g).

Mã nguồn 3.2: Hai conversation độc lập bằng InMemorySaver

```
"""Thread-scoped short-term memory with InMemorySaver."""

import sys

from langchain_core.messages import AIMessage, BaseMessage, HumanMessage
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.graph import END, START, MessagesState, StateGraph


def _find_name(messages: list[BaseMessage]) -> str | None:
    markers = ("t\u00f4i t\u00e0n l\u00e0 ", "t\u00f4i t\u00e0n ")
    for message in reversed(messages):
        if not isinstance(message, HumanMessage):
            continue
        normalized = str(message.content).strip()
        lowered = normalized.lower()
        for marker in markers:
            position = lowered.find(marker)
            if position >= 0:
                value = normalized[position + len(marker) :].strip(" !?,")
                if value and "g\u00e0" not in value.lower():
                    return value
    return None


def deterministic_model(state: MessagesState) -> dict:
    """Replace a real LLM with rules to test memory deterministically."""
    messages = state["messages"]
    latest = str(messages[-1].content).lower()
```

```

if "t\u00f4i t\u00e2n g\u00ec" in latest:
    name = _find_name(messages[:-1])
    answer = f"B\u00e2n t\u00e2n {name}." if name else "T\u00f4i ch\u01b0a bi\u1ebft t\u00e2n b\u00e2n"
    \u2192 "."
else:
    name = _find_name(messages)
    answer = f"Ch\u00e2n {name}!" if name else "T\u00f4i \u0111\u00e3 nh\u00e2n \u0111\u01b0\u1ee3c"
    \u2192 tin nh\u00e2n."
return {"messages": [AIMessage(content=answer)]}

def build_graph(checkpointer: InMemorySaver | None = None):
    builder = StateGraph(MessagesState)
    builder.add_node("call_model", deterministic_model)
    builder.add_edge(START, "call_model")
    builder.add_edge("call_model", END)
    return builder.compile(checkpointer=checkpointer or InMemorySaver())

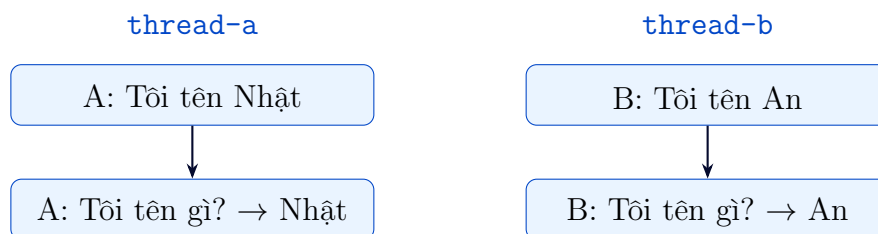
def ask(graph, thread_id: str, text: str) -> str:
    config = {"configurable": {"thread_id": thread_id}}
    result = graph.invoke(
        {"messages": [{"role": "user", "content": text}]},
        config,
    )
    return str(result["messages"][-1].content)

if __name__ == "__main__":
    if hasattr(sys.stdout, "reconfigure"):
        sys.stdout.reconfigure(encoding="utf-8")
    chat_graph = build_graph()
    print(ask(chat_graph, "thread-a", "T\u00f4i t\u00e2n Nh\u00e2n."))
    print(ask(chat_graph, "thread-b", "T\u00f4i t\u00e2n An."))
    print(ask(chat_graph, "thread-a", "T\u00f4i t\u00e2n g\u00ec?"))
    print(ask(chat_graph, "thread-b", "T\u00f4i t\u00e2n g\u00ec?"))

```

Model quyết định trong listing tìm tên trong các HumanMessage cũ. Nó không nhầm thay LLM; nó làm oracle ổn định để test đúng cơ chế memory.

3.2. Dòng thời gian của hai thread



Hình 3.1: Cùng graph và checkpointer, state được phân vùng bởi thread.

Nguồn: Tác giả tự dựng.

3.3. Chỉ gửi message mới

Frontend gọi:

Mã nguồn 3.3: Input đúng khi backend sở hữu history

```
graph.invoke(
    {"messages": [{"role": "user", "content": new_message}]},
    {"configurable": {"thread_id": thread_id}},
)
```

Nếu frontend vừa gửi lại toàn bộ history vừa giữ nguyên checkpointer, message cũ sẽ bị append lần nữa. Đây là lỗi duplicate context khó nhìn thấy khi UI chỉ render từng bong bóng một lần.

Cảnh báo: Thread ID không phải quyền truy cập

Biết một UUID không chứng minh người gọi sở hữu thread. API phải lấy `user_id` từ cơ chế xác thực và kiểm tra ownership trước khi invoke hoặc đọc state. Không tin `user_id` do client tự khai trong production.

3.4. InMemorySaver dùng đúng chỗ

In-memory thích hợp cho học và test. Restart process làm mất state; nhiều worker có các vùng RAM riêng. Backend lâu dài cần checkpointer database. LangGraph cung cấp các package checkpointer độc lập cho SQLite, PostgreSQL, MongoDB, Redis và các hệ khác (LangChain 2026b).

3.5. Đọc state để debug

Mã nguồn 3.4: Đọc snapshot hiện tại của thread

```
config = {"configurable": {"thread_id": "thread-a"}}
snapshot = graph.get_state(config)

for message in snapshot.values["messages"]:
    print(message.type, message.content)
```

`get_state` hữu ích để debug và dựng màn hình history, nhưng endpoint phải kiểm tra owner. Không trả toàn bộ internal state nếu trong đó có tool result hoặc dữ liệu nhạy cảm.

Điểm tự kiểm tra: Lab 25 phút

Chạy `short_term_memory.py`; thêm thread C chưa khai tên và hỏi “Tôi tên gì?”. Sau đó cố ý dùng `thread-a` cho người B để quan sát leak. Viết một đoạn giải thích vì sao UUID không thay thế authorization. Minh chứng đạt: có test tự động cho A, B và C.

4. QUẢN LÝ CONTEXT DÀI

Persistence trả lời “ta lưu được gì?”. Context management trả lời “lượt gọi này model được đọc gì?”. Hai câu hỏi phải tách rời. Một thread có thể lưu 2.000 message để audit nhưng chỉ gửi summary và phần gần nhất vào LLM.

4.1. Vì sao context dài không miễn phí?

Message tích lũy làm tăng token đầu vào, độ trễ và chi phí. Ngay cả khi chưa vượt context window, thông tin cũ hoặc lạc đề có thể làm model phân tâm. Tài liệu LangChain liệt kê ba nhóm xử lý phổ biến: trim, xóa và tóm tắt (LangChain 2026g).

Định nghĩa: Token budget

Token budget là giới hạn dành cho toàn bộ input: system instruction, long-term memory được truy xuất, summary, message gần, tài liệu RAG và tool result. Không nên dành 100% context window cho history vì model còn cần không gian sinh output.

4.2. Trim là tạo một view ngắn

Mã nguồn 4.1: Trim bản input, không xóa storage

```
"""Long-context strategies: trimming and a simple running summary."""

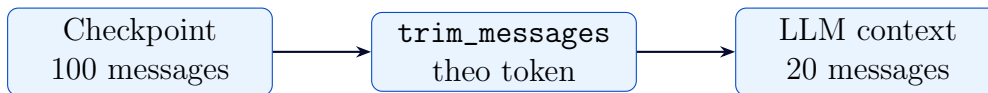
from dataclasses import dataclass

from langchain_core.messages import BaseMessage
from langchain_core.messages.utils import (
    count_tokens_approximately,
    trim_messages,
)

def context_for_model(
    messages: list[BaseMessage],
    *,
    max_tokens: int = 2_000,
) -> list[BaseMessage]:
    """Create a short model view without deleting stored history."""
    return trim_messages(
        messages,
        strategy="last",
        token_counter=count_tokens_approximately,
        max_tokens=max_tokens,
        start_on="human",
        end_on=("human", "tool", "ai"),
        include_system=True,
    )
```

Hàm `context_for_model` nhận danh sách đầy đủ và trả một danh sách mới để gọi model.

Nếu node chỉ dùng danh sách mới nhưng trả về duy nhất AI response, checkpointer vẫn có history đầy đủ. Đây thường là mặc định dễ vận hành nhất.



Hình 4.1: Trim tạo view cho model nhưng có thể giữ storage đầy đủ.

Nguồn: Tác giả tự dựng.

4.3. Không cắt message một cách mù quáng

History hợp lệ thường phải bắt đầu ở ranh giới human message. Nếu assistant tạo tool call, tool result tương ứng phải đi cùng. Vì vậy nên dùng utility hiểu message type thay vì cắt list bằng `messages[-20:]`. Tài liệu chính thức minh họa `trim_messages` với strategy `last`, token counter và ràng buộc đầu/cuối (LangChain 2026a).

Cảnh báo: Hai kiểu “xóa”

Cắt bản input chỉ làm model không thấy phần cũ ở lượt này. Trả `RemoveMessage` vào state có thể xóa message khỏi state checkpointed. Hãy ghi rõ chủ đích, đặc biệt nếu hệ thống cần audit hoặc cho người dùng tải lịch sử.

4.4. Phân bổ context có chủ đích

Một budget minh họa 32.000 token:

Bảng 4.1: Ví dụ phân bổ context; số thực tế tùy model và use case

Thành phần	Token	Ghi chú
System instruction và policy	2.000	Luôn giữ, version rõ ràng.
Long-term user memory	1.000	Chỉ memory liên quan.
Running summary	3.000	Phần hội thoại cũ đã nén.
Recent messages	10.000	Giữ cập hội thoại gần nhất.
RAG/tool context	8.000	Có thể bằng 0 nếu không dùng.
Output reserve và khoảng an toàn	8.000	Không đưa vào input.

Đây không phải công thức chung. Điều cần giữ là invariant: budget được đo, có khoảng an toàn và được test với cuộc trò chuyện dài.

Điểm tự kiểm tra: Lab 20 phút

Tạo 30 `HumanMessage`/`AIMessage`, đặt một fact quan trọng ở đầu. Trim với ba mức token. Ghi lại số message còn lại và kiểm tra fact có biến mất không. Sau đó đề xuất

nơi khác để giữ fact: summary thread hoặc long-term Store. Học viên phải giải thích lựa chọn, không chỉ chạy code.

5. RUNNING SUMMARY

Trim rẻ và dễ đoán nhưng có thể làm mất fact cũ. Running summary nén phần hội thoại cũ thành một đoạn state, rồi kết hợp summary với các message gần nhất. LangChain có `SummarizationMiddleware`; LangGraph cũng cho phép tự xây node tóm tắt (LangChain 2026a,g).

5.1. State mở rộng

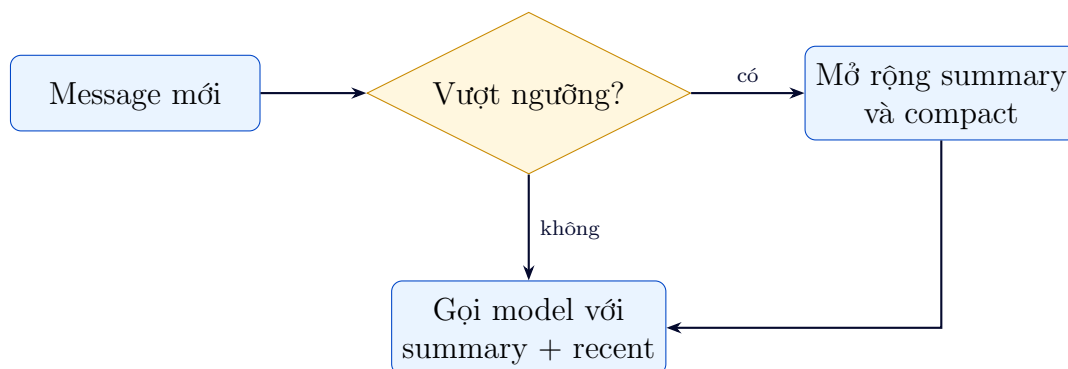
Mã nguồn 5.1: State có thêm summary

```
from langgraph.graph import MessagesState

class ChatState(MessagesState):
    summary: str
```

Summary vẫn là short-term memory nếu nó nằm trong state của một thread. Nó không tự trở thành long-term user profile.

5.2. Chu trình tóm tắt tăng dần



Hình 5.1: Running summary chỉ chạy khi history vượt ngưỡng.

Nguồn: Tác giả tự dựng.

Lần đầu, summarizer nhận phần cũ và yêu cầu tạo summary. Lần sau, nó nhận summary hiện tại cùng các message mới chưa nén và yêu cầu mở rộng summary. Sau đó ứng dụng giữ một số message gần nhất nguyên văn.

5.3. Summary tốt phải giữ điều gì?

- mục tiêu hiện tại và ràng buộc của user;

- quyết định đã thống nhất và lý do;
- việc đã thử, output hoặc lỗi quan trọng;
- tên entity, ID nghiệp vụ cần cho bước sau;
- câu hỏi hoặc action item chưa hoàn thành.

Nên bỏ lời chào, câu lặp, diễn giải dài đã hết hiệu lực và nội dung không liên quan. Summary phải được coi là dữ liệu suy diễn: model có thể tóm tắt sai. Với quyết định nghiệp vụ quan trọng, lưu cấu trúc riêng hoặc nguồn gốc message thay vì chỉ dựa vào văn bản summary.

5.4. Ví dụ compact quyết định

Mã nguồn 5.2: Mô phỏng compact không cần LLM

```
class CompactedConversation:
    summary: str
    recent_messages: list[dict[str, str]]

def compact_deterministically(
    messages: list[dict[str, str]],
    *,
    keep_recent: int = 4,
) -> CompactedConversation:
    """Demonstrate compaction with rules; this is not an LLM summarizer."""
    if len(messages) <= keep_recent:
        return CompactedConversation("", messages.copy())

    old_messages = messages[:-keep_recent]
    facts: list[str] = []
    for message in old_messages:
        content = message["content"].strip()
        if message["role"] == "user" and content:
            facts.append(f"User said: {content}")
    summary = " ".join(facts[-5:])
    return CompactedConversation(summary, messages[-keep_recent:])
```

Hàm minh họa chỉ gom các câu user cũ, vì mục tiêu là quan sát cấu trúc output. Trong production, thay nó bằng model tóm tắt nhỏ hơn và prompt có schema. Một output tốt có thể là JSON:

Mã nguồn 5.3: Schema summary dễ đánh giá

```
summary = {
    "goal": "Integrate LangGraph into the LLM Gateway",
    "decisions": ["Use one thread_id per conversation"],
    "facts": ["User name is Nhat"],
    "open_questions": ["Choose SQLite or PostgreSQL"],
}
```


5.5. Khi nào summary chạy?

Ba lựa chọn:

- theo số message: đơn giản nhưng không phản ánh độ dài thật;
- theo token: sát context window hơn;
- background: giảm độ trễ hot path nhưng phức tạp về consistency.

Với prototype, trigger theo token ở trước model call là đủ. Với production tải lớn, cần nhắc job nền và version summary để tránh hai worker ghi đè.

Điểm tự kiểm tra: Lab 25 phút

Cho một lịch sử 12 message có tên user, mục tiêu dự án, một quyết định bị thay đổi và một câu hỏi chưa giải quyết. Viết prompt summary yêu cầu bốn trường JSON ở trên. Chạy hai lần và chấm: có giữ quyết định mới nhất không; có loại quyết định cũ không; có bịa fact không; recent messages có còn nguyên văn không.

6. LONG-TERM MEMORY THEO USER

Long-term memory sống qua nhiều conversation. LangGraph tổ chức memory dạng tài liệu JSON trong Store; mỗi item có namespace và key (LangChain 2026f). Checkpointer và Store giải quyết hai trục khác nhau.

Bảng 6.1: Checkpointer và Store

	Checkpointer	Store
Phạm vi	Một thread	Nhiều thread hoặc toàn ứng dụng
Khóa thường dùng	<code>thread_id</code>	namespace chứa <code>user_id</code>
Dữ liệu	messages, summary, workflow state	profile, preference, quy tắc, memory chọn lọc
Ví dụ	Câu vừa nói trong chat này	User thích giải thích từng bước

6.1. Namespace và key

Mã nguồn 6.1: Profile key-value trong InMemoryStore

```

"""Key-value long-term memory namespaced by user_id."""

import sys
from typing import Any

from langgraph.store.memory import InMemoryStore

def profile_namespace(user_id: str) -> tuple[str, str, str]:
    return ("users", user_id, "profile")

def save_profile(store: InMemoryStore, user_id: str, profile: dict[str, Any]) -> None:
    store.put(profile_namespace(user_id), "main", profile)

def load_profile(store: InMemoryStore, user_id: str) -> dict[str, Any]:
    item = store.get(profile_namespace(user_id), "main")
    return dict(item.value) if item else {}

if __name__ == "__main__":
    if hasattr(sys.stdout, "reconfigure"):
        sys.stdout.reconfigure(encoding="utf-8")
    memory_store = InMemoryStore()
    save_profile(
        memory_store,
        "user-nhat",
        {

```

```

        "name": "Nh\u1eadt",
        "language": "Vietnamese",
        "answer_style": "t\u1eebng b\u01b0\u1edbc",
    },
)
print(load_profile(memory_store, "user-nhat"))

```

Namespace ("users", user_id, "profile") giống đường dẫn logic. Key main chọn document profile chính. Thiết kế namespace phải nhất quán và luôn gồm tenant/user phù hợp.

6.2. Memory nào đáng lưu?

- **Semantic memory:** fact ổn định như tên, ngôn ngữ, preference.
- **Episodic memory:** trải nghiệm hoặc việc đã xảy ra có ích cho lần sau.
- **Procedural memory:** quy tắc hành vi, ví dụ luôn đưa code trước giải thích.

Phiên bản đầu chỉ nên lưu fact và preference khi user yêu cầu rõ: “Hãy nhớ...”. Điều này tạo kỳ vọng minh bạch và giảm việc thu thập quá mức.

Cảnh báo: Không lưu secret

Không ghi API key, access token, mật khẩu, dữ liệu thanh toán hoặc thông tin nhạy cảm không cần thiết vào memory. Cần có chức năng xem, sửa, quên memory và chính sách retention.

6.3. Kết hợp thread và user

Mã nguồn 6.2: ChatService kết hợp checkpointer và Store

```

"""Combine a short-term checkpointer and long-term Store."""

from dataclasses import dataclass
from typing import Any

from langchain_core.messages import AIMessage
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.graph import END, START, MessagesState, StateGraph
from langgraph.runtime import Runtime
from langgraph.store.memory import InMemoryStore

from long_term_memory import load_profile, save_profile
from short_term_memory import _find_name

@dataclass(frozen=True)
class UserContext:
    user_id: str

class ChatService:
    def __init__(
        self,

```

```

*,
checkpointer: InMemorySaver | None = None,
store: InMemoryStore | None = None,
) -> None:
    self.checkpointer = checkpointer or InMemorySaver()
    self.store = store or InMemoryStore()

    def call_model(state: MessagesState, runtime: Runtime[UserContext]) -> dict:
        user_id = runtime.context.user_id
        messages = state["messages"]
        latest = str(messages[-1].content)
        lowered = latest.lower()

        if "h\u00e3y nh\u01b0" in lowered and (name := _find_name(messages)):
            profile = load_profile(self.store, user_id)
            profile["name"] = name
            save_profile(self.store, user_id, profile)
            answer = f"T\u00f4i s\u01b0 nh\u01b0 b\u01ea1n t\u00e2n {name}."
        elif "t\u00f4i g\u00ec" in lowered:
            thread_name = _find_name(messages[:-1])
            profile_name = load_profile(self.store, user_id).get("name")
            name = thread_name or profile_name
            answer = f"B\u01ea1n t\u00e2n {name}." if name else "T\u00f4i ch\u01b0a bi\u01bft t\u00e2n
        b\u01ea1n."
        else:
            answer = "T\u00f4i \u0111\u00e3 nh\u01b0 \u0111\u01b0\u01ee3c tin nh\u01b0."

        return {"messages": [AIMessage(content=answer)]}

    builder = StateGraph(MessagesState, context_schema=UserContext)
    builder.add_node("call_model", call_model)
    builder.add_edge(START, "call_model")
    builder.add_edge("call_model", END)
    self.graph = builder.compile(
        checkpointer=self.checkpointer,
        store=self.store,
    )

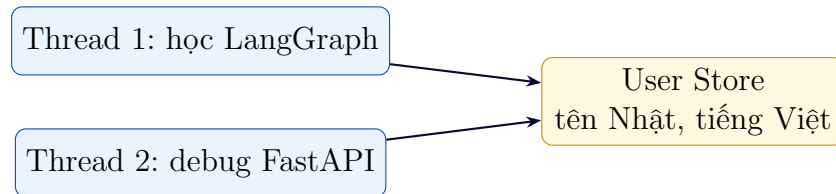
    def ask(self, *, user_id: str, thread_id: str, message: str) -> str:
        config = {"configurable": {"thread_id": thread_id}}
        result = self.graph.invoke(
            {"messages": [{"role": "user", "content": message}]},
            config,
            context=UserContext(user_id=user_id),
        )
        return str(result["messages"][-1].content)

    def history(self, thread_id: str) -> list[dict[str, Any]]:
        config = {"configurable": {"thread_id": thread_id}}
        snapshot = self.graph.get_state(config)
        return [
            {"role": message.type, "content": str(message.content)}
            for message in snapshot.values.get("messages", [])
        ]

    def delete_thread(self, thread_id: str) -> None:
        self.checkpointer.delete_thread(thread_id)

```

Runtime[UserContext] mang user_id vào node mà không biến nó thành lời nhắc cho model. Node dùng thread history trước; nếu không thấy tên trong thread mới tìm profile trong Store. Khi user nói “Hãy nhớ”, profile được cập nhật.



Hình 6.1: Nhiều thread của cùng user truy cập một namespace memory.

Nguồn: Tác giả tự dựng.

6.4. Key-value hay semantic search?

Profile nhỏ và có schema nên dùng key-value. Khi có hàng trăm memory dạng văn bản tự do, semantic search có thể lấy vài item liên quan theo câu hỏi. Store hỗ trợ cấu hình embedding; Qdrant cũng có thể đảm nhiệm lớp vector. Không thêm embedding chỉ vì framework cho phép: retrieval phải giải quyết một nhu cầu tìm kiếm đã đo được.

Điểm tự kiểm tra: Lab 25 phút

User A ở thread 1 nói “Hãy nhớ tôi tên Nhật”, rồi hỏi tên ở thread 2. User B hỏi cùng câu ở thread 3. Kết quả đúng: A được nhớ xuyên thread; B không nhận memory của A. Thêm test cho cả hai chiều.

7. PERSISTENCE VÀ RANH GIỚI PRODUCTION

In-memory chứng minh logic nhưng không chứng minh độ bền. Production cần làm rõ vòng đời dữ liệu, concurrency, encryption, backup, retention và quyền truy cập.

7.1. Chọn backend theo giai đoạn

Bảng 7.1: Lộ trình backend cho memory

Backend	Phù hợp	Giới hạn hoặc yêu cầu
InMemorySaver/ Store	Unit test, workshop	Mất khi restart; không chia sẻ giữa worker.
SQLite	App local, demo bền	Một file; cần quản lý connection và contention.
PostgreSQL	Web production	Cần migration/setup, pool, backup và quan sát DB.
Redis/MongoDB	Kiến trúc có sẵn	Chọn vì yêu cầu vận hành, không vì xu hướng.

LangGraph tách các implementation thành package. SQLite được mô tả phù hợp cho local workflow; PostgreSQL phù hợp production và có bản async (LangChain 2026e).

7.2. Async backend đi với async graph

FastAPI handler thường async. Nếu graph gọi model async qua `ainvoke`, nên dùng `AsyncSqliteSaver` hoặc `AsyncPostgresSaver`. Tránh gọi database đồng bộ nặng trên event loop. Khởi tạo connection pool trong lifespan và đóng khi application shutdown.

Mã nguồn 7.1: Khung lifecycle cho checkpointer production

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    async with AsyncPostgresSaver.from_conn_string(DB_URI) as saver:
        # Run saver.setup() in the initial migration/deploy step.
        app.state.chat_graph = build_graph(checkpointer=saver)
    yield
```

7.3. Ownership là invariant tầng ứng dụng

Checkpointing tìm thread theo ID; nó không biết chính sách business của bạn. Trước mọi thao tác:

1. xác thực request và lấy user ID tin cậy từ token/session;
2. tra owner của thread trong bảng conversation;
3. chỉ sau khi owner khớp mới invoke hoặc get state;
4. log metadata cần thiết nhưng không log toàn bộ prompt mặc định.

Ví dụ: Bảng conversation tối thiểu

```
CREATE TABLE conversations (
  thread_id UUID PRIMARY KEY,
  owner_user_id UUID NOT NULL,
  title TEXT,
  created_at TIMESTAMPTZ NOT NULL,
  deleted_at TIMESTAMPTZ
);
```

7.4. Retention, xóa và quyền được quên

Nút “Xóa cuộc trò chuyện” không chỉ xóa bong bóng trên Streamlit. Nó phải xóa hoặc đánh dấu xóa conversation metadata và checkpoint tương ứng. Long-term profile là phạm vi khác: xóa thread không tự xóa memory user; giao diện nên có thao tác riêng “Quên thông tin này”.

7.5. Checklist vận hành

- ID không đoán được nhưng vẫn có authorization.
- Timeout, retry và idempotency cho model call.
- Không retry mù quáng bước đã có side effect.
- Mã hóa khi truyền và khi lưu theo yêu cầu dữ liệu.
- Backup/restore được diễn tập.
- Metric: token input, số message context, thời gian retrieve, thời gian model.
- Test nhiều worker và restart process.

Điểm tự kiểm tra: Bài thiết kế 20 phút

Chọn SQLite hoặc PostgreSQL cho hai tình huống: chatbot chạy trên laptop một người; dịch vụ có 10.000 user và nhiều worker. Nêu ba lý do, một rủi ro và một test

restart cho mỗi lựa chọn.

8. TÍCH HỢP VÀO LLM GATEWAY

Dự án đích có Streamlit, FastAPI và hai provider DeepSeek/Gemini. Thay đổi quan trọng không phải thêm một import LangGraph; đó là chuyển hợp đồng từ “prompt đơn” sang “message mới trong một conversation có owner”.

8.1. Hợp đồng API đề xuất

Mã nguồn 8.1: Tạo conversation và gửi message

```
POST /conversations
{"user_id": "user-nhat"}

201 Created
{"thread_id": "c20f...", "user_id": "user-nhat"}

POST /conversations/c20f.../messages
{
  "user_id": "user-nhat",
  "message": "What is my name?"
}
```

Frontend giữ `thread_id` hiện tại. Nút “Cuộc trò chuyện mới” gọi endpoint tạo thread. Khi gửi message, frontend không gửi lại full history; backend/checkpointer sở hữu history.

8.2. API chạy được với model quyết định

Mã nguồn 8.2: FastAPI conversation resource

```
"""Minimal conversation API with a per-thread ownership check."""

from uuid import uuid4

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

from chat_service import ChatService

app = FastAPI(title="LangGraph Memory Lab")
chat_service = ChatService()
thread_owners: dict[str, str] = {}

class CreateConversationRequest(BaseModel):
    user_id: str = Field(min_length=1, max_length=100)

class MessageRequest(BaseModel):
    user_id: str = Field(min_length=1, max_length=100)
    message: str = Field(min_length=1, max_length=100_000)
```

```
def require_owner(thread_id: str, user_id: str) -> None:
    owner = thread_owners.get(thread_id)
    if owner is None:
        raise HTTPException(status_code=404, detail="Conversation not found.")
    if owner != user_id:
        raise HTTPException(status_code=403, detail="Conversation owner mismatch.")

@app.post("/conversations", status_code=201)
def create_conversation(payload: CreateConversationRequest):
    thread_id = str(uuid4())
    thread_owners[thread_id] = payload.user_id
    return {"thread_id": thread_id, "user_id": payload.user_id}

@app.post("/conversations/{thread_id}/messages")
def send_message(thread_id: str, payload: MessageRequest):
    require_owner(thread_id, payload.user_id)
    answer = chat_service.ask(
        user_id=payload.user_id,
        thread_id=thread_id,
        message=payload.message.strip(),
    )
    return {"thread_id": thread_id, "answer": answer}

@app.get("/conversations/{thread_id}/messages")
def get_messages(thread_id: str, user_id: str):
    require_owner(thread_id, user_id)
    return {"thread_id": thread_id, "messages": chat_service.history(thread_id)}

@app.delete("/conversations/{thread_id}", status_code=204)
def delete_conversation(thread_id: str, user_id: str):
    require_owner(thread_id, user_id)
    chat_service.delete_thread(thread_id)
    del thread_owners[thread_id]
```

Dictionary `thread_owners` chỉ dành cho lab; production dùng bảng database và user ID từ authentication middleware. Điều đáng học là thứ tự `require_owner` đứng trước đọc hoặc invoke graph.

8.3. Adapter model thật

LangGraph không bắt buộc dùng wrapper model của LangChain, nhưng wrapper chat giúp hai provider nhận cùng kiểu message. Listing sau tạo model dựa trên provider:

Mã nguồn 8.3: Adapter DeepSeek và Gemini

```
"""Real model adapters. This module needs the 'providers' extra and API keys."""

import os

from langchain_core.language_models.chat_models import BaseChatModel

def build_model(provider: str) -> BaseChatModel:
    if provider == "deepseek":
        from langchain_openai import ChatOpenAI
```

```
return ChatOpenAI(
    api_key=os.environ["DEEPSEEK_API"],
    base_url=os.getenv("BASE_URL", "https://api.deepseek.com"),
    model=os.getenv("DEEPSEEK_MODEL", "deepseek-chat"),
    max_tokens=int(os.getenv("MAX_OUTPUT_TOKENS", "4096")),
)

if provider == "gemini":
    from langchain_google_genai import ChatGoogleGenerativeAI

    return ChatGoogleGenerativeAI(
        google_api_key=os.environ["GEMINI_API_KEY"],
        model=os.getenv("GEMINI_MODEL_NAME", "gemini-2.5-flash"),
    )

raise ValueError(f"Unsupported provider: {provider}")
```

Sau đó node thật rất ngắn:

Mã nguồn 8.4: Thay model quyết định bằng chat model thật

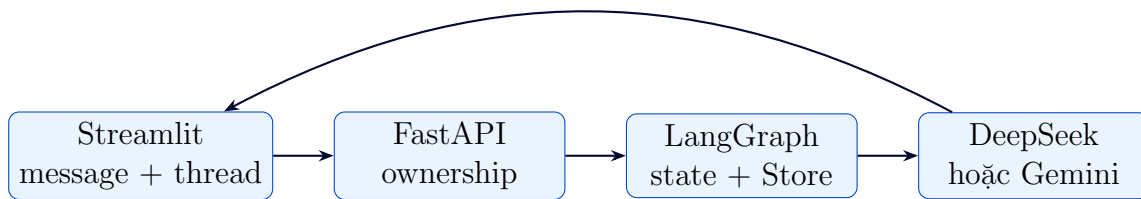
```
async def call_model(state: MessagesState):
    model_messages = context_for_model(
        state["messages"],
        max_tokens=8_000,
    )
    response = await model.ainvoke(model_messages)
    return {"messages": [response]}
```

Không tạo một graph mới cho mỗi request. Model client, checkpointer và compiled graph nên sống theo application lifespan. Provider có thể là trường runtime/context hoặc chọn qua một node, nhưng state schema và invariant thread không đổi.

8.4. System prompt, profile và context

Thứ tự chuẩn bị input đề xuất:

1. nạp policy/system instruction;
2. lấy một số long-term memory liên quan theo user;
3. đọc running summary của thread;
4. trim recent messages theo budget còn lại;
5. gọi đúng provider;
6. append assistant response vào state.



Hình 8.1: Ranh giới thành phần trong LLM Gateway có memory.

Nguồn: Tác giả tự dựng.

8.5. Migration theo lát mỏng

1. Giữ model giả; thêm conversation API và test isolation.
2. Thay model giả bằng một provider; chưa thêm summary.
3. Thêm trim và metric token.
4. Thêm SQLite/PostgreSQL; test restart.
5. Chỉ sau đó thêm long-term memory opt-in.

Cảnh báo: Không gộp mọi thay đổi một lần

Nếu đồng thời đổi API, provider wrapper, database, summary và user memory, khi câu trả lời sai ta không biết lỗi ở context, persistence hay model. Mỗi lát migration cần một invariant và test riêng.

Điểm tự kiểm tra: Lab dự án 30 phút

Chạy uv run uvicorn fastapi_app:app -app-dir code -reload. Tạo hai conversation bằng Swagger, gửi tên khác nhau, đọc history và thử dùng user B đọc thread A. Mình chứng đạt: status 403 xuất hiện trước khi graph state bị đọc.

9. KIỂM THỬ, DEBUG VÀ QUAN SÁT

Một chatbot “có vẻ nhớ” chưa đủ. Ta cần test invariant không phụ thuộc cách LLM diễn đạt. Vì vậy test memory nên dùng model quyết định hoặc fake chat model; test provider thật là lớp integration riêng.

9.1. Bốn invariant cốt lõi

1. Cùng thread: message sau đọc được message trước.
2. Khác thread: short-term state không đi qua.
3. Cùng user, khác thread: long-term memory có thể đi qua.
4. Khác user: long-term memory tuyệt đối không đi qua.

Mã nguồn 9.1: Test short-term và long-term isolation

```
from uuid import uuid4

from langgraph.store.memory import InMemoryStore

from chat_service import ChatService
from long_term_memory import load_profile, save_profile
from short_term_memory import ask, build_graph


def test_two_threads_do_not_mix_names():
    graph = build_graph()
    thread_a = str(uuid4())
    thread_b = str(uuid4())

    ask(graph, thread_a, "T\u00f4i t\u00e2n Nh\u1eadt.")
    ask(graph, thread_b, "T\u00f4i t\u00e2n An.")

    assert ask(graph, thread_a, "T\u00f4i t\u00e2n g\u00ec?") == "B\u1ea1n t\u00e2n Nh\u1eadt."
    assert ask(graph, thread_b, "T\u00f4i t\u00e2n g\u00ec?") == "B\u1ea1n t\u00e2n An."


def test_profile_is_separated_by_user_id():
    store = InMemoryStore()
    save_profile(store, "user-a", {"name": "Nh\u1eadt"})
    save_profile(store, "user-b", {"name": "An"})

    assert load_profile(store, "user-a")["name"] == "Nh\u1eadt"
    assert load_profile(store, "user-b")["name"] == "An"


def test_long_term_memory_crosses_threads_for_same_user():
    service = ChatService()
    service.ask(
        user_id="user-nhat",
        thread_id=str(uuid4()),
```

```

        message="H\u00e3y nh\u1ed7 t\u00f4i t\u00e2n l\u00e0 Nh\u1eadt.",
    )

    answer = service.ask(
        user_id="user-nhat",
        thread_id=str(uuid4()),
        message="T\u00f4i t\u00e2n g\u00ec?",
    )
    assert answer == "B\u1ea1n t\u00e2n Nh\u1eadt."

def test_long_term_memory_does_not_cross_users():
    service = ChatService()
    service.ask(
        user_id="user-a",
        thread_id=str(uuid4()),
        message="H\u00e3y nh\u1ed7 t\u00f4i t\u00e2n l\u00e0 Nh\u1eadt.",
    )

    answer = service.ask(
        user_id="user-b",
        thread_id=str(uuid4()),
        message="T\u00f4i t\u00e2n g\u00ec?",
    )
    assert answer == "T\u00f4i ch\u01b0a bi\u1ebft t\u00e2n b\u1ea1n."

```

9.2. Test API, không chỉ test graph

Graph đúng vẫn có thể bị lỗi dữ liệu nếu route quên ownership check. Vì vậy cần test endpoint bằng hai user:

Mã nguồn 9.2: Test ownership tại ranh giới FastAPI

```

from fastapi.testclient import TestClient

from fastapi_app import app

def test_conversation_flow_and_ownership():
    with TestClient(app) as client:
        created = client.post("/conversations", json={"user_id": "user-a"})
        assert created.status_code == 201
        thread_id = created.json()["thread_id"]

        response = client.post(
            f"/conversations/{thread_id}/messages",
            json={"user_id": "user-a", "message": "T\u00f4i t\u00e2n Nh\u1eadt."},
        )
        assert response.status_code == 200

        forbidden = client.get(
            f"/conversations/{thread_id}/messages",
            params={"user_id": "user-b"},
        )
        assert forbidden.status_code == 403

def test_unknown_thread_returns_404():
    with TestClient(app) as client:
        response = client.get(
            "/conversations/not-found/messages",
            params={"user_id": "user-a"},

```

```
)
assert response.status_code == 404
```

Một test production nên lấy identity từ authentication fixture; payload không được tự quyết định user đang đăng nhập.

9.3. Failure lab

Hãy cố ý tạo và sửa bốn lỗi:

- dùng một `thread_id` cố định cho mọi user;
- frontend gửi full history lần message mới;
- restart InMemorySaver rồi mong history còn;
- namespace Store thiếu `user_id`.

Với mỗi lỗi, ghi ba dòng: triệu chứng; nguyên nhân; test hồi quy. Đây là artifact quan trọng hơn ảnh chụp một lần chatbot trả lời đúng.

9.4. Quan sát gì trong log và metric?

Bảng 9.1: Tín hiệu vận hành không cần log toàn bộ nội dung chat

Tín hiệu	Mục đích	Cảnh báo
Hash/ID thread và user	Trace request	Tránh log PII thô.
Số message lưu/gửi	Phát hiện duplicate	Không đồng nhất hai con số.
Token input/output	Chi phí và context	Cần phân tách provider/model.
Thời gian check-point/store/model	Tìm bottleneck	Đo từng phase.
Số lần summary	Phát hiện trigger sai	Summary quá thường gây tốn phí.
Lỗi authorization	Theo dõi tấn công/lỗi UI	Không log token xác thực.

9.5. Đánh giá chất lượng summary

Chuẩn bị dataset các conversation nhỏ có ground truth:

- fact phải giữ;
- fact đã bị user sửa;

- thông tin phải loại;
- action item chưa hoàn thành.

Chấm summary theo recall fact quan trọng, ưu tiên thông tin mới, không bịa và độ ngắn. Không chỉ đánh giá bằng cảm giác một đoạn văn “nghe hay”.

Điểm tự kiểm tra: Lab 20 phút

Chạy toàn bộ pytest. Sau đó cố ý bỏ `user_id` khỏi namespace profile và quan sát test nào đáng lẽ phải fail. Nếu không test nào fail, bổ sung test trước khi sửa code. Đây là thực hành red-green-refactor.

10. DỰ ÁN CUỐI KHÓA: CHATBOT NHIỀU CUỘC HỘI THOẠI

10.1. Đề bài

Nâng cấp LLM Gateway FastAPI/Streamlit để một user có nhiều conversation độc lập, lịch sử sống qua restart, context dài được kiểm soát và long-term memory chỉ được ghi khi user yêu cầu.

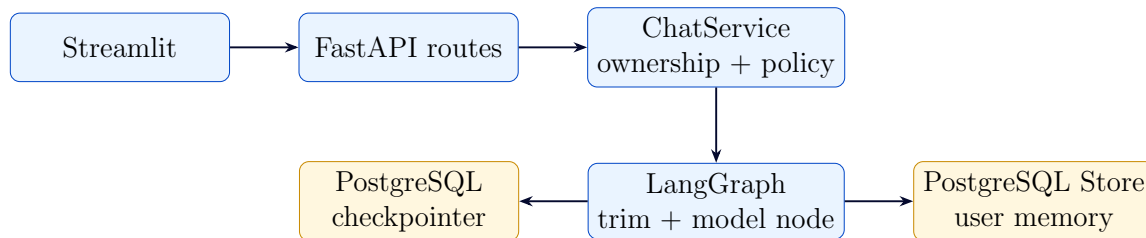
10.2. Yêu cầu chức năng

1. Tạo, liệt kê, đổi tên và xóa conversation.
2. Gửi message mới bằng `thread_id`; frontend không gửi full history.
3. Đọc history sau khi refresh trình duyệt.
4. Chọn DeepSeek hoặc Gemini mà không trộn state.
5. Trim theo token; có running summary khi vượt ngưỡng.
6. Lệnh “Hãy nhớ...” ghi profile; lệnh “Hãy quên...” xóa item.
7. User A không đọc hoặc ghi thread/memory của user B.

10.3. Yêu cầu phi chức năng

- Không log API key hoặc full prompt mặc định.
- Có timeout và thông báo lỗi provider có nghĩa.
- Có test restart persistence.
- Có metric token/context và thời gian gọi model.
- Có README mô tả data flow và retention.

10.4. Kiến trúc tham chiếu



Hình 10.1: Kiến trúc tham chiếu cho dự án cuối khóa.

Nguồn: Tác giả tự dựng.

10.5. Kế hoạch thực hiện theo lát

Bảng 10.1: Các mốc dự án có thể kiểm chứng

Mốc	Phạm vi	Điều kiện hoàn thành
1	Conversation resource	Hai thread bằng model giả; test isolation xanh.
2	Provider thật	Một provider chạy; lỗi API được map rõ.
3	Durable checkpoint	Restart server vẫn đọc được history.
4	Context policy	Có token metric, trim và test conversation dài.
5	User memory	Opt-in save/forget; cross-user test xanh.
6	UI và release QA	Tạo/xóa/chuyển thread; demo và README hoàn chỉnh.

10.6. Demo 5 phút bắt buộc

1. Tạo thread A, nói tên Nhật.
2. Tạo thread B, chứng minh short-term history A không xuất hiện.
3. Ở A nói “Hãy nhớ tôi tên Nhật”; tạo thread C và hỏi tên.
4. Đăng nhập user B hoặc dùng fixture B, chứng minh không đọc được thread A.
5. Restart backend, chứng minh durable history còn.
6. Hiển thị metric cho một request dài.

10.7. Definition of Done

Dự án chưa hoàn thành nếu chỉ có một lần demo thành công. Hoàn thành nghĩa là code, test, migration, data policy và tài liệu khớp nhau; các failure case đã được quan sát; không còn secret trong repo.

Điểm tự kiểm tra: Tự phản biện kiến trúc

Học viên phải trả lời: vì sao dùng LangGraph thay vì list/database tự viết; phần nào vẫn thuộc trách nhiệm ứng dụng; nếu bỏ LangGraph thì invariant nào phải tự cài lại; backend nào được chọn và vì sao.

Kết luận

Bộ nhớ hội thoại không bắt đầu ở vector database. Nó bắt đầu ở một hợp đồng đơn giản: mỗi message thuộc đúng conversation; mỗi conversation thuộc đúng user; state được lưu có chủ đích; model chỉ đọc lát context phù hợp. LangGraph cung cấp state graph, checkpointing và Store để biểu diễn hợp đồng ấy, nhưng authorization, retention, schema memory và đánh giá chất lượng vẫn là trách nhiệm của ứng dụng.

Lộ trình an toàn là: Python thuần để hiểu cơ chế; model quyết định để test; InMemorySaver để học thread; database để có durability; trim trước summary; key-value trước semantic memory; provider thật sau khi isolation đã xanh. Khi từng lát có test, việc thêm RAG, tool hoặc agent về sau không làm mất nền móng hội thoại.

Điểm tự kiểm tra: Ba câu phải trả lời được

1. Dữ liệu nào đang được lưu, ở đâu và theo khóa nào?
 2. Lướt gọi model này thực sự nhận những message nào?
 3. Điều gì chứng minh user khác không đọc được state hoặc memory này?
- Nếu ba câu đều có câu trả lời và test quan sát được, hệ thống đã đi đúng hướng.

A. Cài đặt và lệnh chạy

A.1. Cài môi trường

Từ thư mục giáo trình:

```
uv sync --extra dev
uv run pytest -q
```

Chạy từng ví dụ:

```
uv run python code/manual_history.py
uv run python code/graph_basics.py
uv run python code/short_term_memory.py
uv run python code/long_term_memory.py
```

Chạy FastAPI lab:

```
uv run uvicorn fastapi_app:app --app-dir code --reload
```

Mở <http://127.0.0.1:8000/docs> để chạy request bằng Swagger UI.

A.2. Provider thật

```
uv sync --extra dev --extra providers
```

Đặt API key bằng biến môi trường hoặc file `.env` đã bị ignore; không commit secret. Adapter tham khảo nằm ở `code/real_model_adapter.py`.

A.3. Build PDF

Máy có MiKTeX theo cấu hình repository chạy:

```
.\build-latex.ps1
```

Output ở `build/main.pdf`. Build script dùng thư mục MiKTeX riêng của project và không ghi đè các biến profile phổ biến.

A.4. Fallback

Nếu chưa cài được LangGraph, học chương 1 bằng `manual_history.py`; nếu không có API key, chạy toàn bộ test với model quyết định; nếu chưa có PostgreSQL, dùng InMemorySaver để hoàn thiện logic rồi viết test contract cho backend sau.

B. Bài tập tổng hợp và rubric

B.1. Bài tập cấp 1: mô hình tinh thần

1. Vẽ ba lớp UI history, persistent history và model context cho một request.
2. Viết conversation manager bằng list/dictionary cho ba thread.
3. Giải thích bằng 100 từ vì sao model không tự nhớ request trước.

B.2. Bài tập cấp 2: LangGraph memory

4. Mở rộng state có `summary` và `selected_provider`.
5. Viết test 20 lượt chat xen kẽ giữa hai thread.
6. Trim theo token và đo số message model nhận.
7. Thiết kế namespace profile cho tổ chức, user và application.

B.3. Bài tập cấp 3: production

8. Thay InMemorySaver bằng SQLite và chứng minh restart.
9. Viết bảng conversation ownership và endpoint 403.
10. Thêm lệnh xem/quên long-term memory.
11. Viết kế hoạch migration lên PostgreSQL nhiều worker.

B.4. Rubric dự án cuối khóa

Bảng B.1: Rubric 100 điểm cho chatbot có memory

Tiêu chí	Điểm	Minh chứng
Mô hình state/thread/user đúng	15	Sơ đồ, schema và giải thích nhất quán.
Short-term isolation	15	Test nhiều thread, không duplicate message.
Context management	15	Token budget, trim/summary và test dài.
Long-term memory	15	Opt-in save/forget, namespace đúng.
Security và privacy	15	Ownership, secret policy, cross-user test.
Persistence và vận hành	10	Restart, lifecycle, error handling.
Kiểm thử và quan sát	10	Unit/API/integration test, metric có nghĩa.
Tài liệu và demo	5	README, sơ đồ, demo tái lập được.

B.5. Thang phản hồi bốn mức

- **1 – Chưa đạt:** chỉ demo thử công, không tách đúng thread/user.
- **2 – Đang phát triển:** flow chính chạy nhưng thiếu failure test hoặc persistence.
- **3 – Đạt:** các invariant chính có test, context và ownership rõ.
- **4 – Vững:** có đánh giá dài hạn, quan sát, migration và trade-off được chứng minh.

Mẫu phản hồi: một điểm mạnh quan sát được; một khoảng trống gần rubric; một bước sửa cụ thể; một test nhỏ phải hoàn thành trước buổi sau.

C. Thuật ngữ nhanh

Bảng C.1: Thuật ngữ cốt lõi

Thuật ngữ	Nghĩa trong giáo trình
Context window	Giới hạn token model có thể nhận trong một lượt.
Message	Đơn vị hội thoại có role và content.
State	Dữ liệu hiện tại được node đọc và cập nhật.
Node	Hàm hoặc runnable thực hiện một bước của graph.
Edge	Quan hệ điều hướng giữa các node.
Reducer	Quy tắc hợp nhất node output vào state.
Thread	Chuỗi các lượt thuộc cùng một conversation.
<code>thread_id</code>	Định danh tra state/checkpoint của conversation.
Checkpoint	Snapshot state tại một bước thực thi.
Checkpointing	Backend ghi và đọc checkpoint.
Short-term memory	State trong phạm vi một thread.
Store	Kho tài liệu JSON dùng xuyên thread.
Namespace	Tuple phân vùng item trong Store.
Long-term memory	Thông tin user/app sống qua nhiều thread.
Trim	Chọn lát message ngắn hơn cho model input.
Running summary	Bản tóm tắt tăng dần của phần hội thoại cũ.
RAG	Truy xuất kiến thức ngoài tại thời điểm trả lời; không đồng nhất với memory.
Ownership	Quan hệ business xác định user nào được truy cập thread.

Tài liệu tham khảo

- LangChain (2026a). *Add and manage memory*. URL: <https://docs.langchain.com/oss/python/langgraph/add-memory> (truy cập ngày 01/08/2026).
- (2026b). *Checkpoint integration*. URL: <https://docs.langchain.com/oss/python/integrations/checkpointers/index> (truy cập ngày 01/08/2026).
- (2026c). *Install LangGraph*. URL: <https://docs.langchain.com/oss/python/langgraph/install> (truy cập ngày 01/08/2026).
- (2026d). *LangGraph overview*. URL: <https://docs.langchain.com/oss/python/langgraph/overview> (truy cập ngày 01/08/2026).
- (2026e). *LangGraph persistence*. URL: <https://docs.langchain.com/oss/python/langgraph/persistence> (truy cập ngày 01/08/2026).
- (2026f). *Long-term memory*. URL: <https://docs.langchain.com/oss/python/langchain/long-term-memory> (truy cập ngày 01/08/2026).
- (2026g). *Short-term memory*. URL: <https://docs.langchain.com/oss/python/langchain/short-term-memory> (truy cập ngày 01/08/2026).